Rate limits

Limits protect the stations, which report on fixed schedules, from being asked the same question thousands of times a minute. They are generous for any client that respects the cache, and every response says where you stand, so a well-behaved client never has to guess.

Windows

Every plan shares one rule: a key has a rolling minute window on `/v1/weather/current` and a rolling hour window across all endpoints, the size of each depends on the plan, and both counters are reported on every response. A request that finds either window empty is answered with `429 rate_limited` and is not counted.

Header	Meaning
`X-Limit-Minute`	Requests left in the current minute window.
`X-Limit-Hour`	Requests left in the current hour window.
`Retry-After`	Seconds to wait, present only on a `429`.

Windows are rolling, not aligned to the clock: a request made at 12:00:30 leaves the minute window at 12:01:30. A `304` answer to a conditional request counts against neither window, and neither does a `429` or a `5xx`.

Backoff

On a `429` or a `503`, wait for `Retry-After` when present, otherwise double the delay from one second up to thirty, and stop after five attempts. Add a random fraction of a second to every delay so that several clients that failed together do not retry together.

```
async function withBackoff(request, attempts = 5) {
  let delay = 1000
  for (let attempt = 1; ; attempt++) {
    const response = await request()
    if (response.status !== 429 && response.status !== 503) return response
    if (attempt === attempts) return response
    const retryAfter = Number(response.headers.get("retry-after"))
    const wait =
      retryAfter > 0 ? retryAfter * 1000 : delay + Math.random() * 1000
    await new Promise((resolve) => setTimeout(resolve, wait))
    delay = Math.min(delay * 2, 30_000)
```

```
  }
}
```

Caching

Current conditions are cached at the edge for sixty seconds, so a poll faster than that returns the same reading and spends the minute window for nothing. Forecasts are cached until the next model run, and history responses for a day. Every cached response carries an `ETag`; sending it back as `If-None-Match` turns an unchanged answer into a `304` that costs nothing.

Request costs

Not every request weighs the same against the hour window. Heavier queries cost more, so a client that pages through history with small pages pays more than one that asks for large pages.

Endpoint	Cost	Notes
Current conditions	1	Also counted in the minute window.
Hourly forecast	2	Regardless of `hours` and `fields`.
Daily forecast	2	
Observation history	5 per page	A page of 1,000 rows costs the same as one of 10.
Station directory	1 per page	
Station detail	1	Cache it for a day.
Weather alerts	1	Business and Enterprise plans.

`X-Limit-Hour` reports the remaining hour budget in cost units, not in requests; `X-Limit-Minute` counts requests, since only one endpoint has a minute window.

Plans compared

The plans differ in window size, forecast horizon, history retention and support. The table has seven columns, so the paginated outputs put it on a landscape page of its own.

Plan	Stations	Minute window	Hour window	Forecast hours	History	Support
Community	3	60	600	24	7 days	Forum
Team	25	120	6,000	48	90 days	Email, 2 days
Business	250	600	60,000	48	1 year	Email, 1 day
Enterprise	Custom	Custom	Custom	48	Custom	Named contact

Stations counts the distinct stations a project's keys may read, not the number of keys; a project may hold up to ten keys. The minute window applies per key, and the hour window applies to the project as a whole, so it is shared by every key the project holds.

Changing plans

An upgrade takes effect within a minute and widens the current windows in place, so a client that is being limited recovers without waiting for the hour to roll over. A downgrade takes effect at the end of the billing period. History older than the new retention stops being served at that point but is kept for thirty days, so a downgrade made by mistake can be reversed without loss.

Change	Takes effect	Windows	History
Upgrade	Within a minute	Widened in place	Older history becomes available at once
Downgrade	End of the billing period	Narrowed at the next hour window	Kept for thirty days, then trimmed
Cancellation	End of the billing period	Live keys stop; test keys keep working	Exportable for thirty days

Appendix: measuring your own usage

Log the two limit headers with every response and chart the minimum per minute. A flat line near zero means a client is polling faster than the sixty-second cache refreshes; slow it down rather than raising the plan. One log line per response is enough:

```
2026-09-22T06:00:01Z GET /v1/weather/current station=OSL-01 status=200 minute=57
hour=5931 request=req_7f3a9c2e
```

A query over a day of such lines shows where the window runs lowest:

```
SELECT date_trunc('minute', at) AS minute, min(minute_left) AS worst
FROM northlight_requests
WHERE at > now() - interval '1 day'
GROUP BY 1
ORDER BY worst
LIMIT 20;
```

Three patterns account for almost every limit incident:

- **Synchronized polling.** Every instance of a service wakes on the same second. Offset instances by a few seconds each.
- **Retry storms.** A `503` from one station is retried at once by every instance. Use the backoff above; the jitter is not optional.
- **Missing conditional requests.** A poll that ignores `ETag` pays for every unchanged reading. Send `If-None-Match`.

> **Retired keys keep counting**
>
> Requests made with a retired key are rejected but still counted against the project for one hour, so rotate keys during a quiet period and switch every consumer before the old key is revoked.